Kamya Hari

LinkedIn: Kamya Hari
GitHub: kamyahari
Google Scholar

Email: khari8@gatech.edu
Mobile: +1-678-656-3954
Atlanta,USA

Education

Georgia Institute of Technology — Aug 2024 - May 2026
Master of Science in Electrical and Computer Engineering — *GPA:3.91/4.0*
Courses: *Advanced VLSI, Advanced Programming Techniques, Digital Systems Test, DSP Hardware Design, Parallel Programming for FPGA, GPU Hardware Software, Advanced Computer Architecture*

Sri Sivasubramaniya Nadar College of Engineering — Aug 2019 - May 2023
Bachelor of Engineering - Electronics and Communication Engineering — *GPA: 9.3/10*
Courses: *Digital System Design, Principles of VLSI Design, Computer Architecture and Organization, Machine Learning*

Skills

- **Languages**: Python, C++, Verilog, SystemVerilog, VHDL, CUDA, MATLAB, Tcl
- **Frameworks**: Scikit, Scipy, Keras, PyTorch, OpenMP, OpenGL, OpenMPI, HuggingFace, Ollama, UVM
- **Software Tools**: Simulink, Jupyter Notebook, Google Colab, Git, Vitis HLS, Vivado, Model Sim, Innovus, Virtuoso, NSight

Experience

Language, Intelligence and Thought Lab, Georgia Tech — Atlanta, Georgia
Graduate Research Assistant — *Aug 2024- Present*

- Built GPU-accelerated pipelines to automate semantic labeling of narrative stimuli using open-source LLMs.
- Developing encoding models and analyzing model responses with respect to language-selective brain regions in task-based fMRI data.

SHARC Lab, Georgia Tech — Atlanta, Georgia
Research Assistant — *Aug 2025- Present*

- Contributing to OmniSim, a cycle-accurate simulator for C/RTL co-simulation for HLS.
- Debugging RTL waveforms for AXI modeling mismatches.
- Improving Vitis HLS-generated RTL readability and interpretability using LLM-based variable renaming and comments.

Speech and Language Technology Lab, SSN College of Engineering — Chennai, India
Research Assistant — *Aug 2023- June-2024*

- Classified sentences from EEG signals and converted them to speech.
- Collected EEG data using Emotiv Epoc X 14-channel headset.
- Used 2D CNNs for feature extraction, Genetic Algorithms for feature selection, and Decision Trees for classification; achieved 41.79% accuracy for 64-class sentence-level classification. (SoTA).

Projects

- **GPU Cache and Tensor Core Simulation (C++, MacSim)**: Implemented GPU compute and tensor core simulation in MacSim, modeling instruction latencies, register dependencies, and parallel execution. Extended warp scheduling algorithms (Round Robin, Greedy-Then-Oldest, Cache-Conscious Wavefront Scheduling) to improve cache locality and warp utilization, and analyzed NVBit-generated traces (HMMA/FFMA) to evaluate performance trade-offs across compute and tensor workloads.
- **Two-Level Cache Hierarchy Simulator (C++, Python, Linux)**: Designed and implemented a configurable L1–L2 cache simulator supporting WBWA/WTWNA policies, MIP/LIP insertion, and +1/Markov/Hybrid prefetchers. Automated large-scale design space exploration across cache size, associativity, block size, and predictor table depth using Bash + Python. Achieved up to 30.8% reduction in L1 Average Access Time via hybrid prefetching and memory hierarchy tuning; evaluated cost-performance tradeoffs using custom tag/metadata storage models.
- **Parallel Bitonic Sort on GPU (C++, CUDA, Nsight Compute)**: Implemented and optimized a CUDA-based Bitonic Sort leveraging shared memory, pinned memory, asynchoronous transfers and loop unrolling techniques. Profiled with Nsight Compute to tune memory throughput and occupancy, achieving 1250 million elements/sec (222× CPU speedup, with constraints).
- **Branch Divergence Detection using SASS Analysis (Python)**: Built a static analysis tool to detect warp-level branch divergence by constructing def-use tables and CFGs from NVIDIA SASS assembly. Identified thread-ID–dependent branches and loop-induced divergence via iterative dataflow analysis.
- **Neuro-Symbolic Kernel Optimization in FPGA (C++, Vitis HLS)**: Designing and optimizing FPGA-based kernels for neuro-symbolic AI workloads - one layer of ResNet(circular) and circular convolution (symbolic reasoning). Used HLS to implement compute- and memory-efficient pipelines using loop unrolling, data reuse, AXI burst transfers, and dataflow parallelism. Acheived a speedup of 10x for the neural kernel and 50x for the symbolic kernel. GitHub.
- **CORDIC-Based Sine and Cosine Calculator (VHDL, ModelSim, Vivado)**: Implemented a sine and cosine calculator in VHDL using the CORDIC algorithm, leveraging iterative shift-add operations. Verified functional correctness through simulation and waveform analysis, and synthesized the design on a Xilinx FPGA for area and timing evaluation. Optimized area and power for the design using pipelining.